Full Attention Strikes Back: Transferring Full Attention into Sparse within Hundred Training Steps

Yanke Zhou · Yiduo Li · Hanlin Tang · Maohua Li · Kan Liu · Lan Tao · Lin Qu · Yuan Yao · Xiaoxing Ma

ABSTRACT

Long-context inference in large language models is bottlenecked by the quadratic cost of full attention. Existing efficient alternatives often rely either on native sparse training or on heuristic token eviction, creating an undesirable trade-off among efficiency, training cost, and accuracy. In this work, we show that full-attention LLMs are already intrinsically sparse and can be transformed into highly sparse models with only minimal adaptation. Our approach is built on three observations: (1) only a small subset of attention heads truly requires full long-context processing; (2) long-range retrieval is governed primarily by a low-dimensional subspace, allowing relevant tokens to be retrieved efficiently with a 16-dimensional indexer; and (3) the useful token budget is strongly query-dependent, making dynamic top-p selection more suitable than fixed top-k sparsification. Based on these insights, we propose RTPurbo*, which retains the full KV cache only for retrieval heads and introduces a lightweight token indexer for sparse attention. By exploiting the model's intrinsic sparsity,* RTPurbo *achieves sparsification with only a few hundred training steps. Experiments on long-context benchmarks and reasoning tasks show that* RTPurbo *preserves near-lossless accuracy while delivering substantial efficiency gains, including up to a 9.36× prefill speedup at 1M context and about a 2.01× decode speedup. These results suggest that strong sparse inference can be obtained from standard full-attention training without expensive native sparse pretraining.*

1 Introduction

Long-context capability has become a core requirement for modern large language models (LLMs), especially for applications such as multi-turn dialogue, long-horizon reasoning,

†Project lead §Corresponding author ♯Work done during internship at Alibaba

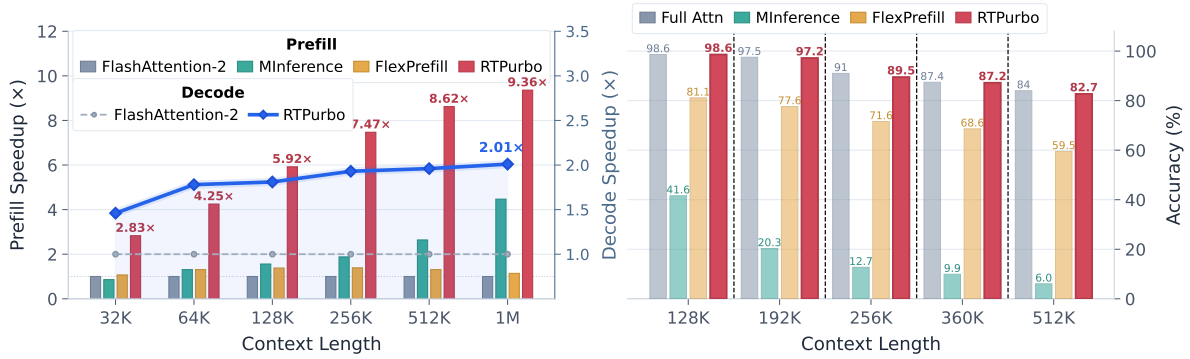


FIGURE 1 A teaser view of the efficiency and accuracy gains of RTPurbo.

and document understanding [3, 8, 22, 30]. However, the cost of full attention grows rapidly with context length, making long-context inference a major efficiency bottleneck. Sparse attention thus emerges as a natural direction for reducing inference cost [26, 32, 33].

Although many recent advances in this area replace standard full attention with more efficient alternatives, such as Kimi Delta Attention [21] and DeepSeek Sparse Attention [5], our study suggests that models trained with full attention already exhibit substantial intrinsic sparsity. Prior work has partially revealed this phenomenon. Specifically, sparsity arises at both the head level and the token level: most heads rely primarily on local information [19, 27, 28], whereas for each query only a small subset of tokens receives substantial attention mass [12, 20, 24]. This observation naturally raises a key question: *What is the minimal surgery required to transform a full-attention model into a highly sparse one while preserving its capabilities?*

We identify three challenges:

- Head selection: a robust metric is needed to identify the heads that genuinely require full-context access.
- Efficient token indexing: a lightweight selector is needed to identify the necessary tokens efficiently.
- Adaptive sparsity: because different queries require different numbers of attended tokens, a static sparsity budget can lead to information loss.

Our method, RTPurbo, is designed to address these challenges with minimal adaptation. The design of RTPurbo is grounded in both LLM interpretability and theoretical analysis. Prior work on inductive heads shows that some heads implement a retrieval mechanism by attending to previously similar tokens [16]. Follow-up work further shows that, in long-context settings, these heads are primarily responsible for remote retrieval, whereas the remaining heads focus on local context [19]. This observation motivates our head-wise design: we retain the full KV cache only for retrieval heads and discard remote tokens for local heads.

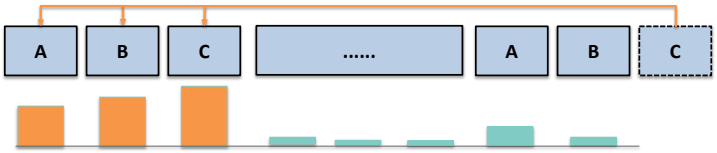


FIGURE 2 Unlike most attention heads that mainly focus on local information, retrieval heads attend to regions that are semantically related to the current query token (e.g. similar pattern), even when those regions are far away in the context.

For retrieval heads, the key challenge is to identify relevant tokens efficiently. Our analysis shows that high-frequency components contribute little to long-range retrieval and can even interfere with it, suggesting that the retrieval process is governed largely by a low-dimensional subspace. This hypothesis is strongly supported by experiments: with our trained low-dimensional projector, we achieve over 90% recall using only 16 dimensions. Moreover, our analysis suggests that a static Top-k selector can fail in certain cases, whereas a Top-p selector better adapts to the attention distribution and yields substantially better accuracy on both reasoning and long-context tasks.

Finally, we find that self-distillation is particularly effective for recovering the performance of the sparsified model. Aligning the sparse model's outputs with those of the original model substantially reduces the risk of overfitting, and only a few hundred training steps (about 1M label tokens) are required for this alignment stage. This result further supports our claim that RTPurbo performs only minimal surgery on the original model.

To the best of our knowledge, RTPurbo is the first method to achieve such near-lossless compression with lightweight continual training. Coupled with our custom sparse kernels, RTPurbo delivers up to a 9.36× speedup in prefill and a 2.01× speedup in decoding (Figure 1). Importantly, the sparsification paradigm of RTPurbo remains highly interpretable. More broadly, our results highlight an overlooked point for full-attention models: *even without native sparse training, a fully trained model can be sparsified with very small additional cost while preserving strong performance*. This finding suggests that full-attention training remains a highly competitive and practical choice.

2 Insight Behind RTPurbo

2.1 Head Specialization as a Natural Prior for Sparse Attention

Recent studies suggest that attention heads in pretrained LLMs are not homogeneous, but instead specialize into distinct functional roles. In particular, prior work has shown that only a small subset of heads is responsible for retrieving distant relevant content, while many others mainly process local information [19, 28]. We refer to this subset as *retrieval heads*. Their characteristic behavior is to place strong attention on earlier context

surrounding semantically related content, thereby exhibiting an information-retrieval pattern, as illustrated in Figure 2.

This observation provides an important design motivation for our method: *we can naturally exploit the sparsity structure that the model has already formed.* Concretely, we retain the full KV cache only for retrieval heads, while for the remaining heads, which are already intrinsically sparse, we can safely discard remote tokens.

2.2 RoPE Induces a Compressible Geometry for Retrieval Heads

Retrieval heads should assign high attention to semantically related tokens even when they are far apart.

However, this property of retrieval heads appears, at first glance, to be in tension with RoPE [18]. For a query token at position m and a key token at position n with dimension $d = 2D$, RoPE injects position through a rotation matrix:

$$R_i(m) = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix}, \qquad q_m = R(m)q,\ k_n = R(n)k, \tag{1}$$

where $R(m) = \mathrm{diag}(R_1(m), \dots, R_D(m))$, and θ_i decreases with the channel index. The resulting query–key score depends only on the relative offset $\Delta = m - n$:

$$s(m,n) = q_m^\top k_n = \sum_{i=1}^{D} \left[a_i(q,k)\cos(\theta_i\Delta) + b_i(q,k)\sin(\theta_i\Delta) \right], \tag{2}$$

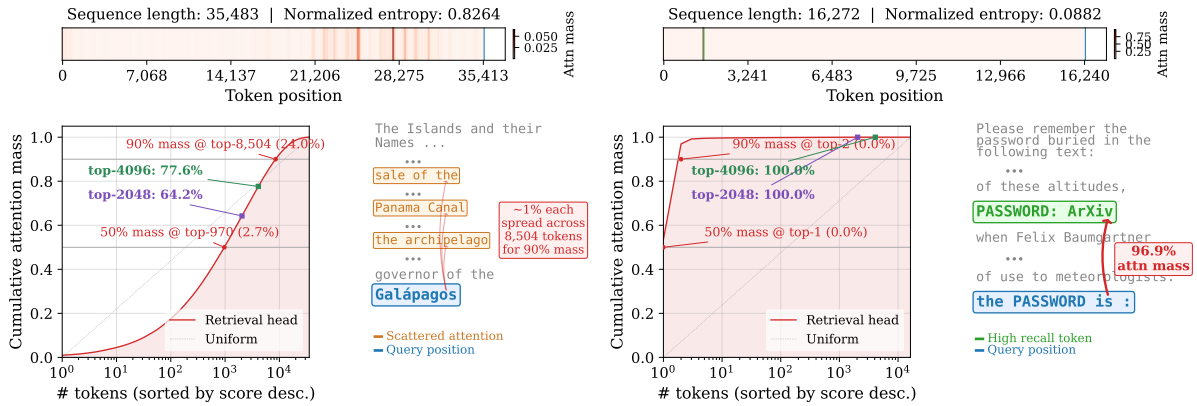
where a_i and b_i are bilinear coefficients induced by the i-th rotary pair. Equation (2) reveals the key distinction directly: high-frequency components vary rapidly with Δ and become distance-sensitive at long range, whereas low-frequency components change smoothly and better preserve retrieval signals. This leads to our second core insight: *we can reconstruct retrieval-head attention in a much lower-dimensional space.*

We therefore use this low-frequency structure as a compact retrieval subspace, enabling low-cost token selection without full-dimensional scoring.

2.3 Retrieval Heads Require Dynamic Thresholding

The remaining question is how many tokens a retrieval head should preserve once relevance can be estimated efficiently. Our findings suggest that this quantity is fundamentally query-dependent. Even within the same retrieval head, different inputs can induce very different patterns: some queries trigger broad retrieval over many distant tokens, while others lock onto only a few key tokens. The required sparsity level is therefore not a fixed attribute of the head; it changes with the query.

Figure 3 illustrates this point. In one case, the query activates a broad semantic field, so the retrieval head must preserve a wide support to recover most of the attention mass. In another, the query only needs to recover a single key fact, so the head is naturally highly concentrated.



(a) Diffuse retrieval triggered by the query token "Galápagos" in a long passage.

(b) Concentrated retrieval in a NIAH query.

FIGURE 3 Retrieval-head behavior is strongly query-dependent. (a) The query token "Galápagos" induces diffuse retrieval over many semantically related earlier tokens: about 8k tokens are needed to recover 90%– attention mass, while top-4k recovers only about 75%. (b) For a needle-in-a-haystack query, retrieval is highly concentrated: two tokens recover 96.6% attention mass, whereas top-4k retains many unnecessary tokens.

Method	Tokens Recall	Attn Mass
top-2k	2048	64.2%
top-4k	4096	77.6%
top-16k	16384	93.8%
top-p 0.9	8504	90.0%
Full Attn	35483	100%

TABLE 1 Fixed top-k trades recall for sparsity: top-16k computes about 8k extra tokens than top-p, but recovers only 3.8% more attention mass.

This is exactly where fixed-budget rules such as top-k sampling become problematic. When k is too small, diffuse queries recover too little attention mass and the approximation becomes inaccurate. When k is too large, the retained set is no longer sparse enough and much of the extra computation is wasted. Table 1 makes this trade-off concrete: top-16k recovers only 3.8% more attention mass than dynamic top-p, but requires computing about 8k additional tokens. The issue is therefore not choosing a better global k; any fixed k is mismatched to the query-dependent nature of retrieval heads.

3 Method

We introduce RTPurbo, a head-wise attention framework with precise token-level sparse computation. This section is organized as follows. We first describe how to identify retrieval heads through offline calibration in Section 3.1. We then present our sparse computation pattern in Section 3.2. Next, we describe the two-stage training pipeline required by RTPurbo in Section 3.3. Finally, we describe the hardware-aware decoding kernel in Section 3.4.

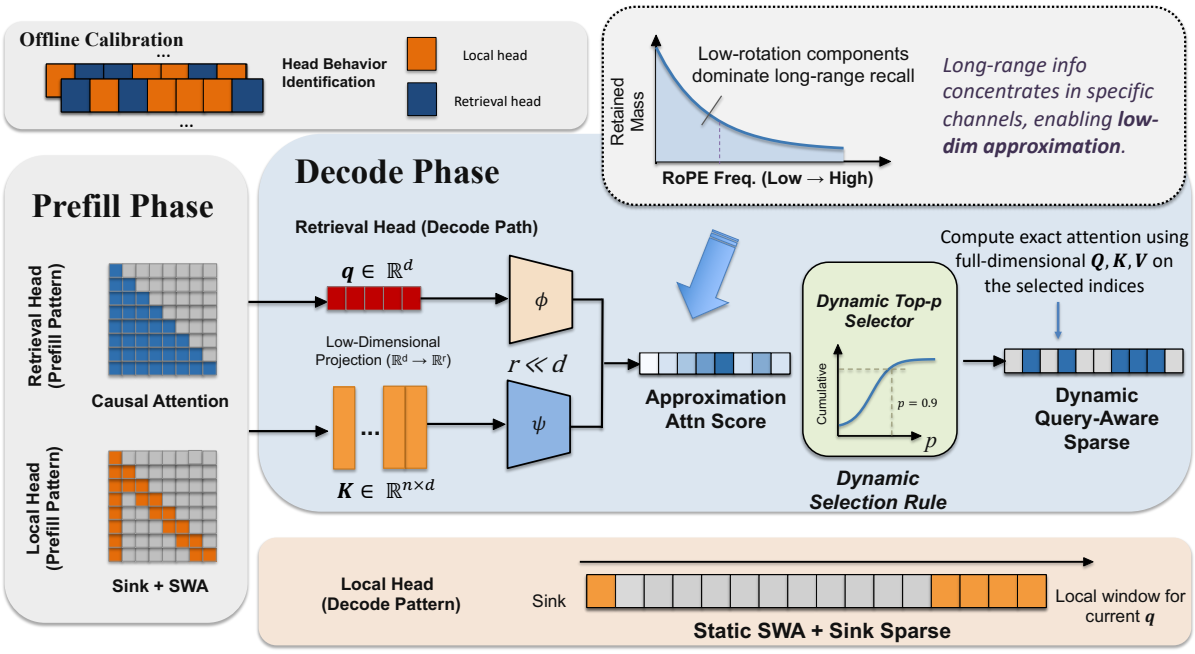


FIGURE 4 Overall architecture of RTPurbo.

3.1 Offline Head-wise Calibration

To identify retrieval heads, we construct a lightweight calibration sequence by inserting an identical "needle" span at both the beginning and the end of a long document sampled from FineWeb [17]. We quantify a head's retrieval capability by measuring the attention mass directed from the later needle to the earlier one. Let $\mathcal{N}_{\text{pre}}$ and $\mathcal{N}_{\text{post}}$ denote the token indices of the earlier and later needle spans, respectively. The retrieval score for head h is compactly defined as:

$$R_h = \frac{1}{|\mathcal{N}_{\text{post}}|} \sum_{t \in \mathcal{N}_{\text{post}}} \sum_{j \in \mathcal{N}_{\text{pre}}} A_h(t, j), \tag{3}$$

where $A_h(t, j)$ represents the normalized attention score (i.e., post-softmax) from token t to token j.

The head retrieval behavior is highly stable and largely input-agnostic. Therefore, in practice, running this calibration on just one single long text sequence is sufficient to robustly score and partition all query heads into a retrieval set $\mathcal{H}_{\text{ret}}$ (top-scoring heads) and a local set $\mathcal{H}_{\text{loc}}$. This partition process is executed only once offline.

3.2 Adaptive Sparse Attention Mechanism

During inference, local heads $h \in \mathcal{H}_{\text{loc}}$ consistently apply a sliding window with attention sinks [27] across both prefill and decode stages. In contrast, retrieval heads $h \in \mathcal{H}_{\text{ret}}$ perform full dense attention during prefill to build the complete KV cache, but switch to a query-aware dynamic sparse selection during decoding. As analyzed in Section 2.2,

high-frequency RoPE components degrade long-range affinity. To circumvent this, we estimate query-key relevance using low-rank projections $W_h^Q, W_h^K \in \mathbb{R}^{r \times d_h}$ ($r \ll d_h$) applied to the features before RoPE injection:

$$s_h(m,n) = (W_h^Q q_{m,h}^{\text{pre}})^\top (W_h^K k_{n,h}^{\text{pre}}), \tag{4}$$

where $q_{m,h}^{\text{pre}}$ and $k_{n,h}^{\text{pre}}$ are the pre-RoPE representations. We then construct a dynamic active set from the projected scores and compute sparse attention as

$$O_h(m) = \sum_{n \in \mathcal{S}_h(m)} \frac{\exp(q_{m,h}^\top k_{n,h}/\sqrt{d_h})}{\sum_{j \in \mathcal{S}_h(m)} \exp(q_{m,h}^\top k_{j,h}/\sqrt{d_h})} v_{n,h}, \qquad \mathcal{S}_h(m) = \text{Top-P}(s_h(m,\cdot), p)\,. \tag{5}$$

In this way, the low-rank pre-RoPE projections serve strictly as an efficient routing mechanism, while the final token generation preserves the complete feature space and exact relative positional geometry. For MQA and GQA models, the resulting sparsity should be interpreted from two perspectives because our head partition is defined over query heads. *Compute sparsity* is measured at the query-head level and can be viewed as the average number of attended tokens over heads. *Memory sparsity* is measured at the KV-head level: for each KV head, the actual retained set is the union of the token sets selected by all query heads mapped to that KV head.

3.3 Low-cost Two-Stage Training

We adopt a lightweight two-stage training pipeline to fully restore model capabilities under the sparse regime. In the first stage, we keep the backbone LLM frozen and independently train the low-dimension projection weights W_h^Q, W_h^K for each retrieval head $h \in \mathcal{H}_{\text{ret}}$. Let $a_h^{\text{full}}(m)$ be the original exact attention distribution and $a_h^{\text{proj}}(m; W_h^Q, W_h^K)$ be the distribution derived from the low-dimensional projected scores. We optimize the projections by minimizing the Kullback-Leibler (KL) divergence between them:

$$\mathcal{L}_{\text{proj}} = \sum_{h \in \mathcal{H}_{\text{ret}}} \text{KL}\Big(a_h^{\text{full}}(m) \,\|\, a_h^{\text{proj}}(m; W_h^Q, W_h^K)\Big)\,. \tag{6}$$

In the second stage, we insert the trained projections, switch to the sparse attention mode, and perform end-to-end self-distillation. The sparse model acts as a student learning to match the dense teacher's next-token predictions. Crucially, compared to standard supervised fine-tuning, *self-distillation bypasses the negative impact of specific dataset distributions, thereby eliminating the tedious need to ablate and tune data mixtures.* To further reduce computational overhead, we align only the top-10 logits of the teacher. Letting $z_{(10)}^{\text{dense}}$ and $z_{(10)}^{\text{sparse}}$ denote the respective logits restricted to these top-10 entries, we minimize:

$$\mathcal{L}_{\text{distill}} = \text{KL}\Big(\text{softmax}(z_{(10)}^{\text{dense}}) \,\|\, \text{softmax}(z_{(10)}^{\text{sparse}})\Big)\,. \tag{7}$$

3.4 Hardware-Aware Fast Top-p Decoding Kernel

We implement the block-wise top-p sparse decoding using a custom GPU kernel that addresses two primary engineering challenges: (1) fast top-p thresholding without expensive sorting, and (2) memory-efficient sparse decoding over long contexts.

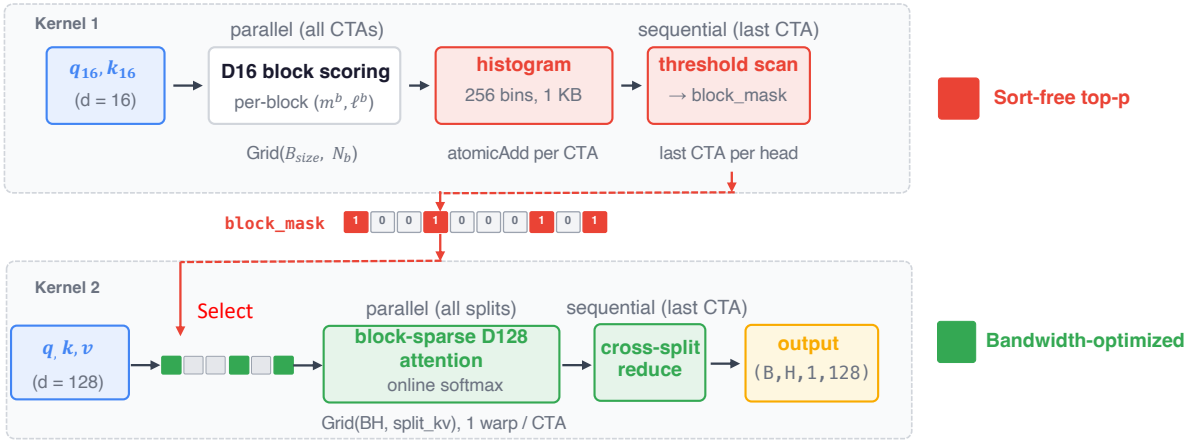


FIGURE 5 Overview of the hardware-aware decoding kernel in RTPurbo.

Sort-free top-p via histogram. We partition compressed K sequence into N_b blocks, where each CTA (Compute Thread Array) computes a low-dimensional attention score for one block and reduces it to a block-level log-sum-exp pair (m_b, ℓ_b). Since commonly used fast sorting methods still incur $O(N_b \log N_b)$ complexity, while binary-search selection requires $O(N_b)$ memory per head, which becomes prohibitive at long context where N_b can exceed 16K, we instead have each CTA atomically deposit ℓ_b into a 256-bin histogram indexed by m_b, which requires only 1 KB per head regardless of sequence length. To avoid an additional kernel launch for the selection phase, each CTA atomically increments a per-head counter upon completion, and the last CTA to finish proceeds to scan the histogram from the highest bin, identifies the score threshold at which the cumulative attention mass reaches p_{top}, and writes a block-level binary mask. This fuses scoring and selection into a single kernel launch with $O(1)$ memory overhead.

Bandwidth-optimized sparse decoding. For long sequences, even sparse attention remains memory-bound because the selected KV blocks can still span tens of thousands of tokens. We address this by designing a single-warp CTA with no shared memory, which keeps all state in registers and allows the SM to maximize concurrent CTAs and thus outstanding memory requests. The inner loop is 2-token unrolled, issuing all K and V loads upfront via vectorized $half2$ instructions so that the subsequent score computation and online-softmax update overlap with in-flight loads. When $B{\times}H$ alone is insufficient to fill the GPU, we further partition the KV range of each head into multiple splits, each handled by a separate CTA, and fuse the cross-split reduction into the last completing CTA via the same atomic-counter technique.

Config	Value
Retrieval head ratio	15%
Sliding window size	8192
Sink tokens	4
Low-dim size	16
Top-p	0.9
Kernel block	64

TABLE 2 RTPurbo config

4 Experiments

All experiments are conducted on NVIDIA H20 GPUs with Python 3.14, CUDA 12.8, and PyTorch 2.8. For accuracy evaluation, we use the `lm-eval` framework [6] as the unified evaluation pipeline.

4.1 Accuracy Evaluation

Benchmarks and Models. We evaluate RTPurbo on two categories of benchmarks. The first category consists of long-context benchmarks, including LongBench [1] and RULER [9], which evaluate overall long-context processing ability. The second category consists of reasoning benchmarks, including AIME24 [14], AIME25 [2], and MMLU-PRO [25], which are used to assess both long-decode performance and the general reasoning ability of the sparsified model. For the first category, we use Qwen3-Coder-30B-A3B, and for the second category, we use Qwen3-30B-A3B-Think, a reasoning-specialized model [29].

Settings. Table 2 summarizes the main configuration of RTPurbo. We also conduct ablation studies on several key design settings of RTPurbo (see Appendix B). For training, we use FineWeb [17] and Dolma 3 Longmimo Mix [15], from which we sample documents with lengths between 32K and 80K tokens. In the first stage, we train the low-dimensional projection parameters. In the second stage, we perform end-to-end training on corpora with an average length of 48K for about only 600 steps. The detailed training procedures are provided in Appendix C.

Baselines. We compare RTPurbo against five representative sparse-attention baselines: RazorAttn [19], Minference [10], FlexPrefill [11], Quest [20], and SnapKV [12]. For each method, we align the evaluation setting with both its official configuration and our own setup as much as possible to ensure a fair comparison. In particular, for RazorAttn we use the same 15% retrieval-head ratio as in RTPurbo. For FlexPrefill, we set the cumulative-attention threshold γ to 0.9 to match our top-p threshold. For Quest, we strictly follow the official implementation, and do not apply sparse attention to the first two layers. Furthermore, to explicitly isolate and evaluate the advantage of our dynamic token budget, we implement a custom baseline that uses a static top-k selection strategy, with k empirically set to 4096.

Results on LongBench and RULER. Table 3 and Table 4 summarize the evaluation results. Methods estimating global attention via recent queries (Minference, SnapKV) degrade significantly on multi-hop tasks (e.g., multi-Q and multi-K) where local context diverges

Longbench	2wiki	hotpot	musique	multi-en	multi-zh	qasper	g-report	qmsum	vcsum	triviaqa	trec	lsht	lcc	repo-p	PR-zh	PR-en	Avg. (%)
Full Attn	42.08	54.64	38.30	52.89	65.99	39.25	31.89	23.77	13.55	89.93	85.50	61.50	35.08	27.61	99.75	99.00	53.80
RazorAttn	40.31	50.92	29.15	53.04	65.90	39.72	31.61	23.59	13.85	89.26	85.00	59.50	35.01	32.05	99.00	99.75	52.98
Minference	40.22	52.21	32.12	52.09	64.92	38.43	31.77	23.60	13.42	51.95	82.50	32.25	34.05	25.76	99.00	100	48.39
FlexPrefill	36.48	52.76	35.71	51.69	65.86	37.44	31.63	24.00	13.10	90.59	80.50	50.25	32.85	24.93	93.50	69.50	49.42
Quest	39.11	53.27	31.64	48.22	62.80	38.27	30.18	23.55	13.24	79.88	81.00	59.00	35.80	24.31	98.83	91.92	50.69
SnapKV	42.56	53.10	38.14	52.70	65.50	39.30	16.11	17.68	11.03	82.03	77.00	57.21	35.00	25.76	99.00	99.75	50.74
RTPurbo																	
w/ top-k	42.56	53.10	36.46	51.75	64.97	40.43	31.76	24.41	13.88	89.10	83.00	55.50	34.41	33.47	99.00	99.00	<u>53.30</u>
w/ top-p	44.49	54.74	35.29	53.34	65.56	40.22	31.92	24.60	14.00	90.85	84.50	60.00	34.56	34.08	99.75	100	54.24

TABLE 3 Accuracy comparison on LongBench. The best average result among sparse-attention methods is shown in bold, and the second-best average result is underlined. Full attention is excluded from this ranking.

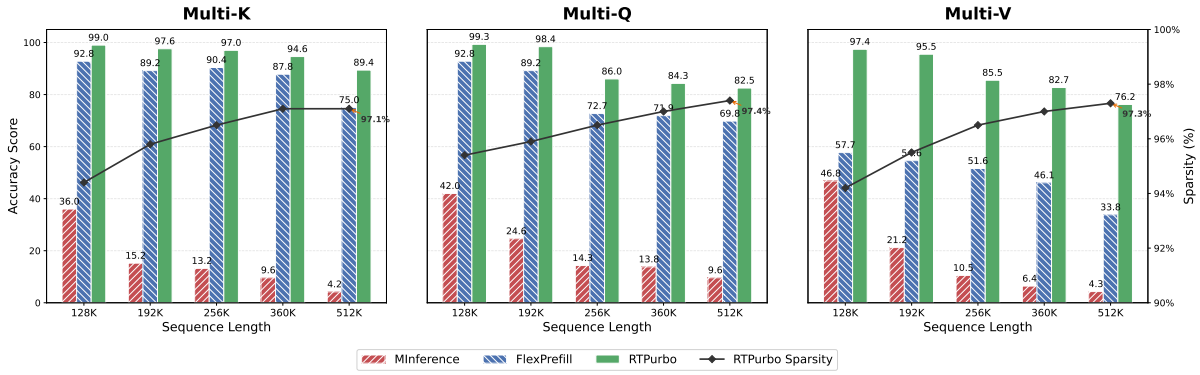


FIGURE 6 Accuracy and sparsity on ultra-long multi-hop tasks (128K–512K). Unlike baselines that collapse at extreme lengths, RTPurbo sustains robust accuracy while achieving high sparsity.

from the full sequence. Similarly, the reliance on adjacent blocks of FlexPrefill causes severe drops on dispersed-evidence tasks like multi-V, while coarse block-level sparsity of Quest yields a general accuracy loss. As a training-free approach, RazorAttn also struggles on retrieval-heavy tasks (e.g., HotpotQA, Musique). Crucially, on long-context benchmarks such as RULER 64K, the fixed-budget variant performs poorly because it recalls too few tokens to preserve sufficient attention mass (see Appendix A.2). Furthermore, we extend our evaluation to ultra-long contexts (up to 512K). As illustrated in Figure 6, while baselines experience catastrophic degradation at extreme lengths, RTPurbo robustly sustains high accuracy. These comparisons confirm that RTPurbo with dynamic top-p selection effectively adapts to varying query complexities, providing a trainable, fine-grained thresholding solution that strictly preserves accuracy.

Results on Reasoning Tasks. Table 5 summarizes the reasoning benchmark results. These tasks exhibit extreme prompt-generation asymmetry: inputs are extremely short (<300 tokens), but generated reasoning traces are massively long (up to 32K tokens on AIME and

Method	CWE	FWE	VT	HotPot	Squad	multi-Q	multi-V	multi-K	niah-S	Avg.
					32K					
Full Attn	81.80	91.47	98.00	64.60	72.63	99.95	98.85	99.66	99.93	89.65
RazorAttn	82.50	90.07	92.04	65.00	71.88	99.60	97.65	99.66	99.80	<u>88.69</u>
Minference	80.88	85.33	95.84	62.00	69.88	97.45	91.35	70.73	98.80	83.58
FlexPrefill	75.26	90.00	96.56	60.40	71.18	98.20	60.30	98.66	100	83.40
Quest	57.26	82.60	97.48	56.80	68.73	85.60	85.20	86.80	90.27	78.97
SnapKV	76.23	89.92	94.23	58.92	69.20	94.00	71.10	97.46	99.86	83.43
RTPurbo										
w/ top-k	82.50	91.00	97.68	58.40	71.78	99.70	98.55	65.53	94.06	84.36
w/ top-p	85.44	91.33	97.84	64.60	72.63	99.95	99.15	99.60	100	90.06
					64K					
Full Attn	65.28	84.00	96.80	63.40	69.83	99.50	97.60	99.66	100	86.23
RazorAttn	65.96	83.80	91.52	62.80	69.33	98.95	95.20	98.73	99.73	<u>85.11</u>
Minference	61.84	82.73	83.48	42.80	29.67	82.25	81.10	40.07	86.53	65.61
FlexPrefill	51.04	80.07	92.72	58.00	68.63	96.35	55.50	97.60	100	77.77
Quest	36.56	63.80	94.80	54.60	62.57	80.40	75.90	78.86	87.93	70.60
SnapKV	60.08	72.00	90.00	47.70	68.15	91.15	60.50	94.00	98.73	75.81
RTPurbo										
w/ top-k	59.92	62.00	69.60	56.00	64.27	98.15	97.60	50.66	76.53	70.53
w/ top-p	65.10	81.40	94.60	62.60	70.03	99.65	97.50	98.60	99.93	85.49

TABLE 4 Accuracy comparison on RULER. The best average results among sparse-attention methods are shown in bold, and the second-best average results are underlined. Full attention is excluded from this ranking.

averaging 10K on MMLU-PRO), shifting the bottleneck entirely to the decoding phase. As shown, RTPurbo with dynamic top-p preserves near-lossless accuracy, perfectly matching the dense baseline on AIME (86.67). On the MMLU-PRO subcategories, it also remains consistently close to the full-attention baseline across all reported subjects, indicating that the sparse model retains strong general reasoning ability even under long decoding.

4.2 Efficiency Evaluation

Sparsity Analysis. During prefill, our sparsity is deterministic: 15% retrieval heads ($\mathcal{H}_{ret}$) perform dense attention, while 85% local heads ($\mathcal{H}_{loc}$) attend only to 4 sink tokens and an 8192-token window. During decode, RTPurbo applies dynamic top-p selection. Table 6 profiles Layer 25 of Qwen3-Coder-30B-A3B, demonstrating that the optimal token budget is highly query-dependent. At 32K, RTPurbo retains just 468.8 active tokens for niah-S but dynamically expands to 2462.1 for multi-K. This 5× variance exposes the inherent flaw of rigid static top-k methods, which inevitably suffer recall failure on complex queries or waste computation on simple ones. By adapting on the fly, we maintain high attention mass (>0.93) with exceptional sparsity (up to 89.2% at 64K).

Furthermore, we extend this decode-stage analysis to ultra-long contexts. As shown in

Method	AIME24	AIME25	MMLU-Bio	MMLU-Bus	MMLU-Chem	MMLU-CS	MMLU-Math	MMLU-Phi	MMLU-Phy
Full Attn	86.67	86.67	89.20	87.40	88.00	86.10	93.60	69.94	90.80
Quest	46.67	46.67	88.28	84.28	84.40	82.68	93.20	69.34	87.70
SnapKV	43.33	46.67	88.14	81.27	76.32	80.49	90.00	66.40	84.42
RTPurbo									
w/ top-k	80.00	80.00	88.80	86.60	51.40	50.49	71.60	51.70	90.60
w/ top-p	86.67	86.67	89.80	87.20	87.80	85.12	93.60	69.54	91.00

TABLE 5 Accuracy comparison on reasoning benchmarks. The best results among sparse-attention methods are shown in bold, and the second-best results are underlined. Full attention is excluded from this ranking.

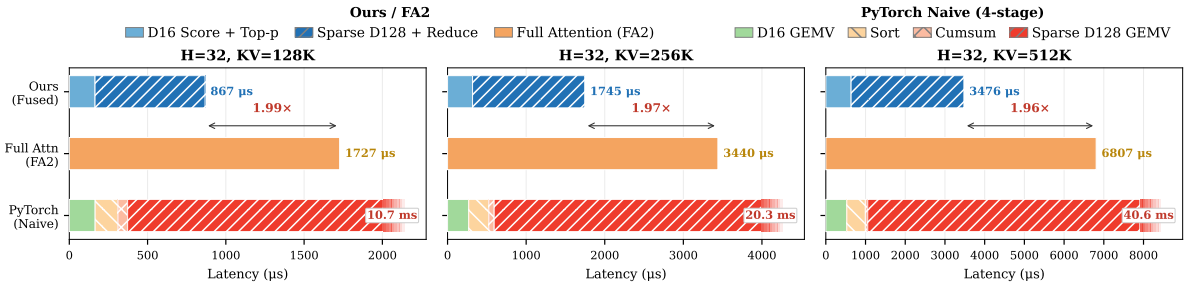


FIGURE 7 Sparse decoding speedup of RTPurbo.

Figure 6, RTPurbo robustly sustains high accuracy while its dynamic thresholding pushes sparsity to over 97.1%[1] at 512K. This confirms that our query-aware mechanism seamlessly extrapolates to extreme context lengths, maximizing efficiency without sacrificing recall.

Runtime Analysis. We measure the speedup of a single attention layer under our sparse execution scheme. The results are shown in the left panel of Figure 1. In the prefill phase, RTPurbo delivers substantial acceleration over FlashAttention-2 (FA2) [4] across all tested context lengths, with speedups increasing from 2.83× at 32K to 9.36× at 1M. It also consistently outperforms the other sparse baselines in long context prefill. In the decode phase, RTPurbo also achieves stable speedups over FA2, improving from about 1.47× at 32K to 2.01× at 1M.

In addition, we benchmark our single-operator top-p decode kernel against both FA2 and a native PyTorch implementation. As shown in Figure 7, our implementation consistently outperforms both baselines, confirming the efficiency of our specially designed top-p decode kernel.

[1] The reported ultra-long context sparsity is calculated as the average compute sparsity across all query heads; actual sparsity levels naturally vary among individual heads. We provide a more detailed analysis of per-head behaviors in Appendix A.

Context	Task	Compute Sparsity	Memory Sparsity	Active Tokens	Attention Mass
32K	niah-S	78.7%	76.2%	468.8	>0.95
	multi-K	77.8%	74.4%	2462.1	>0.96
64K	niah-S	89.2%	87.7%	1126.8	>0.93
	multi-K	88.7%	85.2%	3316.1	>0.94

TABLE 6 Decode-stage dynamic sparsity of our top-p mechanism. The active token budget adaptively scales with task complexity and context length. *Compute/Memory Sparsity* follow the definitions in Section 3.2, while *Active Tokens* and *Attention Mass* denote the dynamically retained KV pairs per retrieval head and their preserved cumulative probability.

5 Conclusion

We show that full-attention LLMs are already intrinsically sparse and can be transformed into efficient sparse inference systems with only minimal adaptation. Based on this view, we propose RTPurbo, a head-wise sparse attention framework built on retrieval/streaming head specialization, low-dimensional retrieval indexing, and dynamic top-p selection.

Empirically, RTPurbo preserves near-lossless accuracy on both long-context and reasoning tasks while delivering substantial prefill and decode speedups. More broadly, our results suggest that native sparse pretraining is not the only path to efficient long-context inference: full-attention models can already support effective sparse execution with lightweight post hoc adaptation.

6 Related work

Block-Sparse Attention. Block-sparse methods reduce long-context cost by selecting only a subset of key–value blocks. QUEST [20] uses query-aware page ranking based on min–max key statistics, while MoBA [13] treats sparse attention as block-level routing. BLASST [31], SpargeAttention [32], and Prism [23] further improve block selection using softmax-contribution estimates, training-free refinement, or spectral criteria. These methods mainly differ in how they estimate block importance at the block level.

Token-wise Attention. Token-wise sparse attention first estimates token relevance and then applies exact attention only to the retained tokens. DSA [5] uses a lightweight learned indexer before top-k selection, FASA [24] exploits the frequency structure induced by RoPE, and SnapKV [12] compresses the KV cache using relevance to recent local queries.

Pattern-Based Sparsity. Another line of work adapts sparsity to head behavior. MInference [10] assigns each head an offline-discovered sparse pattern, while FlexPrefill [11] makes the pattern selection context-aware. DuoAttention [28] and RazorAttention [19] instead partition heads into retrieval and streaming groups and treat them differently. Our

method is most closely related to this line, but further introduces low-dimensional token indexing and dynamic top-p selection for retrieval heads.

References

[1] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL https://aclanthology.org/2024.acl-long.172/.

[2] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions, February 2025. URL https://matharena.ai/.

[3] Gheorghe Comanici et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025. URL https://arxiv.org/abs/2507.06261.

[4] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=mZn2Xyh9Ec.

[5] DeepSeek-AI, Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenhao Xu, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Erhang Li, Fangqi Zhou, Fangyun Lin, Fucong Dai, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoran Wei, Haowei Zhang, Haowen Luo, Haozhe Ji, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, Jialiang Huang, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jingchang Chen, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jinhua Zhu, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexin Huang, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Liang Zhao, Liangsheng Yin, Lihua Guo, Lingxiao Luo, Linwang Ma, Litong Wang, Liyue Zhang, M. S. Di, M. Y Xu, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Panpan Huang, Peixin Cong, Peiyi Wang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, S. H. Liu, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Songyang Zhou, Tao Ni, Tao Yun, Tian Pei, Tian Ye, Tianyuan Yue, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjun Gao, Wentao Zhang, Xi Gao, Xiangwen Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyuan Li, Xu Chen, Xuecheng Su, Xuehai Pan, Xuheng Lin, Xuwei Fu, Y. Q. Wang, Yang Zhang, Yanhong Xu, Yanru Ma, Yao Li, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yiliang Xiong, Ying He, Ying Zhou, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiao Chen,

Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuduan Wang, Yue Gong, Yuhan Wu, Yuheng Zou, Yukun Li, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehua Zhao, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhiyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Zizheng Pan, Zongqing Yao, Bei Feng, Hui Li, J. L. Cai, Jiaqi Ni, Lei Xu, Meng Li, Ning Tian, R. J. Chen, R. L. Jin, S. S. Li, Shuang Zhou, Tianyu Sun, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xinnan Song, Xinyi Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, Dongjie Ji, Jian Liang, Jianzhong Guo, Jin Chen, Leyi Xia, Miaojun Wang, Mingming Li, Peng Zhang, Ruyi Chen, Shangmian Sun, Shaoqing Wu, Shengfeng Ye, T. Wang, W. L. Xiao, Wei An, Xianzu Wang, Xiaowen Sun, Xiaoxiang Wang, Ying Tang, Yukun Zha, Zekai Zhang, Zhe Ju, Zhen Zhang, and Zihua Qu. Deepseek-v3.2: Pushing the frontier of open large language models, 2025. URL https://arxiv.org/abs/2512.02556.

[6] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024. URL https://zenodo.org/records/12608602.

[7] Asma Ghandeharioun, Avi Caciularu, Adam Pearce, Lucas Dixon, and Mor Geva. Patchscopes: A unifying framework for inspecting hidden representations of language models, 2024. URL https://arxiv.org/abs/2401.06102.

[8] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun

Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, September 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL http://dx.doi.org/10.1038/s41586-025-09422-z.

[9] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, and Boris Ginsburg. RULER: What's the real context size of your long-context language models? In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=kIoBbc76Sy.

[10] Huiqiang Jiang, YUCHENG LI, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=fPBACAbqSN.

[11] Xunhao Lai, Jianqiao Lu, Yao Luo, Yiyuan Ma, and Xun Zhou. Flexprefill: A context-aware sparse attention mechanism for efficient long-sequence inference. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=OfjIlbelrT.

[12] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=poE54GOq2l.

[13] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, Yulun Du, Tao Jiang, Chao Hong, Shaowei Liu, Weiran He, Enming Yuan, Yuzhi Wang, Zhiqi Huang, Huan Yuan, Suting Xu, Xinran Xu, Guokun Lai, Yanru Chen, Huabin Zheng, Junjie Yan, Jianlin Su, Yuxin Wu, Yutao Zhang, Zhilin Yang, Xinyu Zhou, Mingxing Zhang, and Jiezhong Qiu. MoBA: Mixture of block attention for long-context LLMs. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=RlqYCpTu1P.

[14] Maxwell-Jia. Aime 2024 dataset, 2024. URL https://huggingface.co/datasets/Maxwell-Jia/AIME_2024.

[15] Team Olmo, :, Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, Robert Berry, Saumya Malik, Saurabh Shah, Scott Geng, Shane Arora, Shashank Gupta, Taira Anderson, Teng Xiao, Tyler Murray, Tyler Romero, Victoria Graf, Akari Asai, Akshita Bhagia, Alexander Wettig, Alisa Liu, Aman Rangapur, Chloe Anastasiades, Costa Huang, Dustin Schwenk, Harsh Trivedi, Ian Magnusson, Jaron Lochner, Jiacheng Liu, Lester James V. Miranda, Maarten Sap, Malia Morgan, Michael Schmitz, Michal Guerquin, Michael Wilson, Regan Huff, Ronan Le Bras, Rui Xin, Rulin Shao, Sam Skjonsberg, Shannon Zejiang Shen, Shuyue Stella Li, Tucker Wilde, Valentina Pyatkin, Will Merrill, Yapei Chang, Yuling Gu, Zhiyuan Zeng, Ashish Sabharwal, Luke Zettlemoyer, Pang Wei Koh, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. Olmo 3, 2026. URL https://arxiv.org/abs/2512.13961.

[16] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads, 2022. URL https://arxiv.org/abs/2209.11895.

[17] Guilherme Penedo, Hynek Kydlíček, Loubna Ben allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro Von Werra, and Thomas Wolf. The fineweb datasets: Decanting the web for the finest text data at scale. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024. URL https://openreview.net/forum?id=n6SCkn2QaG.

[18] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomput.*, 568(C), February 2024. ISSN 0925-2312. doi: 10.1016/j.neucom.2023.127063. URL https://doi.org/10.1016/j.neucom.2023.127063.

[19] Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Danning Ke, Shikuan Hong, Yiwu Yao, and Gongyi Wang. Razorattention: Efficient KV cache compression through retrieval heads. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=tkiZQlL04w.

[20] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: query-aware sparsity for efficient long-context llm inference. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

[21] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, Jiaxi Hu, Fanqing Meng, Chengyin Liu, Xin Men, Songlin Yang, Zhiyuan Li, Wentao Li, Enzhe Lu, Weizhou Liu, Yanru Chen, Weixin Xu, Longhui Yu, Yejie Wang, Yu Fan, Longguang Zhong, Enming Yuan, Dehao Zhang, Yizhi Zhang, T. Y. Liu, Haiming Wang, Shengjun Fang, Weiran He, Shaowei Liu, Yiwei Li, Jianlin Su, Jiezhong Qiu, Bo Pang, Junjie Yan, Zhejun Jiang, Weixiao Huang, Bohong Yin, Jiacheng You, Chu Wei, Zhengtao Wang, Chao Hong, Yutian Chen, Guanduo Chen, Yucheng Wang, Huabin Zheng, Feng Wang, Yibo Liu, Mengnan Dong, Zheng Zhang, Siyuan Pan, Wenhao Wu, Yuhao Wu, Longyu Guan, Jiawen Tao, Guohong Fu, Xinran Xu, Yuzhi Wang, Guokun Lai, Yuxin Wu, Xinyu Zhou, Zhilin Yang, and Yulun Du. Kimi linear: An expressive, efficient attention architecture, 2025. URL https://arxiv.org/abs/2510.26692.

[22] Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Chenxiao Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Qizheng Gu, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yang Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Haoyu Lu, Lijun Lu, Yashuo Luo, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Zeyu

Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Lin Sui, Xinjie Sun, Flood Sung, Yunpeng Tai, Heyi Tang, Jiawen Tao, Qifeng Teng, Chaoran Tian, Chensi Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Si Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Haoning Wu, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Jing Xu, Jing Xu, Junjie Yan, Yuzi Yan, Hao Yang, Xiaofei Yang, Yi Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Siyu Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Shaojie Zheng, Longguang Zhong, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi k2: Open agentic intelligence, 2026. URL https://arxiv.org/abs/2507.20534.

[23] Xinghao Wang, Pengyu Wang, Xiaoran Liu, Fangxu Liu, Jason Chu, Kai Song, and Xipeng Qiu. Prism: Spectral-aware block-sparse attention, 2026. URL https://arxiv.org/abs/2602.08426.

[24] Yifei Wang, Yueqi Wang, Zhenrui Yue, Huimin Zeng, Yong Wang, Ismini Lourentzou, Zhengzhong Tu, Xiangxiang Chu, and Julian McAuley. FASA: FREQUENCY-AWARE SPARSE ATTENTION. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=FnSgecCEwg.

[25] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. Mmlu-pro: a more robust and challenging multi-task language understanding benchmark. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA, 2024. Curran Associates Inc. ISBN 9798331314385.

[26] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=NG7sS51zVF.

[27] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=NG7sS51zVF.

[28] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, junxian guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context LLM inference with retrieval and streaming heads. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=cFu7ze7xUm.

[29] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui

Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL https://arxiv.org/abs/2505.09388.

[30] An Yang, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoyan Huang, Jiandong Jiang, Jianhong Tu, Jianwei Zhang, Jingren Zhou, Junyang Lin, Kai Dang, Kexin Yang, Le Yu, Mei Li, Minmin Sun, Qin Zhu, Rui Men, Tao He, Weijia Xu, Wenbiao Yin, Wenyuan Yu, Xiafei Qiu, Xingzhang Ren, Xinlong Yang, Yong Li, Zhiying Xu, and Zipeng Zhang. Qwen2.5-1m technical report, 2025. URL https://arxiv.org/abs/2501.15383.

[31] Jiayi Yuan, Cameron Shinn, Kai Xu, Jingze Cui, George Klimiashvili, Guangxuan Xiao, Perkz Zheng, Bo Li, Yuxin Zhou, Zhouhai Ye, Weijie You, Tian Zheng, Dominic Brown, Pengbo Wang, Markus Hoehnerbach, Richard Cai, Julien Demouth, John D. Owens, Xia Hu, Song Han, Timmy Liu, and Huizi Mao. Blasst: Dynamic blocked attention sparsity via softmax thresholding. In *Proceedings of the 8th Conference on Machine Learning and Systems*, MLSys '26, 2026. URL https://arxiv.org/abs/2512.12087. Also available as arXiv:2512.12087.

[32] Jintao Zhang, Chendong Xiang, Haofeng Huang, Jia wei, Haocheng Xi, Jun Zhu, and Jianfei Chen. Spargeattention: Accurate and training-free sparse attention accelerating any model inference. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=74c3Wwk8Tc.

[33] Nicolas Zucchet, Francesco D'Angelo, Andrew Kyle Lampinen, and Stephanie C.Y. Chan. The emergence of sparse attention: impact of data distribution and benefits of repetition. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=jMhRbV47pS.

Appendix

A Headwise Analysis of Local/Retrieval Patterns and Retrieval Sparsity

In this section, we analyze the distribution and properties of different head patterns in the two models used in our experiments, namely Qwen3-Coder-30B-A3B and Qwen3-30B-A3B-Think, to better understand their multi-head attention behaviors.

A.1 Headwise Distribution.

In this section, we analyze the distribution of retrieval scores across all query heads in Qwen3-Coder-30B-A3B and Qwen3-30B-A3B-Think.

Retrieval score distribution. Figure 8 presents the per-head retrieval score heatmaps for both models. We make the following observations:

- *Heads with strong retrieval ability are relatively few.* Most query heads receive only small retrieval scores, indicating limited long-range recall ability, while only a relatively small subset of heads consistently exhibits strong retrieval behavior. This suggests that the

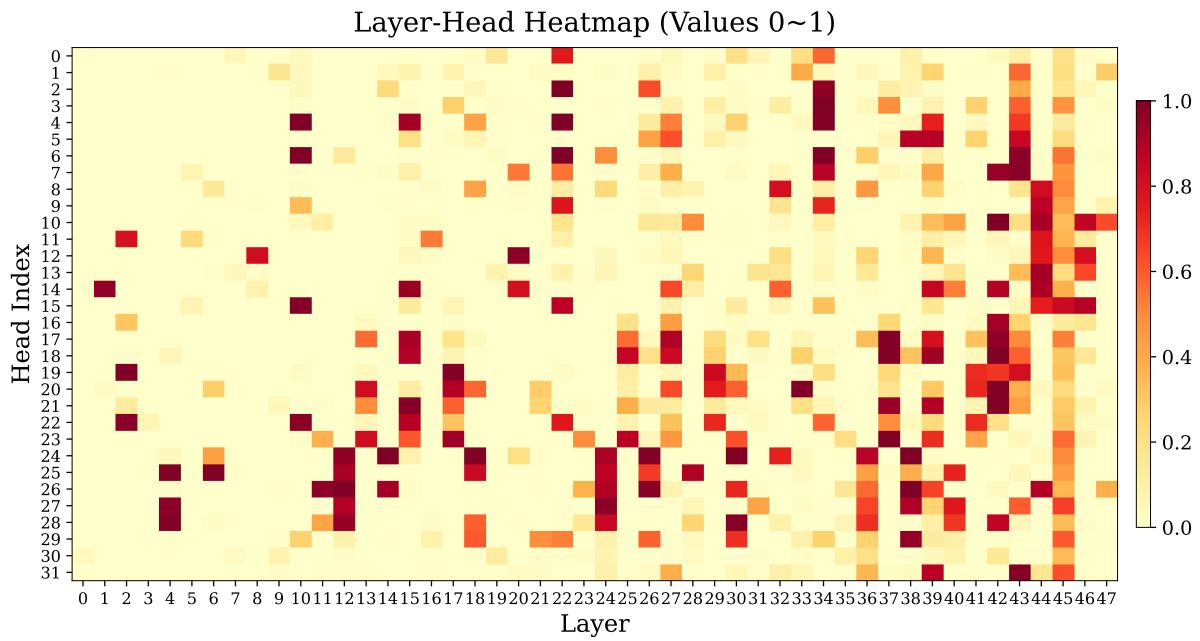


FIGURE 8 Head-wise retrieval scores of all query heads in Qwen3-Coder-30B-A3B. A higher score indicates stronger long-range retrieval ability.

capacity for long-range token recall is concentrated in a minority of heads, whereas the majority primarily rely on local context or sink tokens for information processing.

- *Retrieval heads concentrate in later layers.* High-scoring retrieval heads are distributed highly unevenly across layers, appearing almost exclusively in the latter half of the model. This is consistent with the layerwise computation pattern of LLMs: early layers are primarily responsible for local contextualization, where token representations are still evolving; later layers produce more stable, semantically rich representations that provide the necessary foundation for reliable long-range token retrieval [7].
- *Head behavior is highly stable and input-agnostic.* The retrieval scores of individual heads remain highly consistent across different input documents, confirming our assumption in Section 3.1 that running offline calibration on a single long sequence is sufficient to robustly partition all query heads into retrieval and local sets.

Comparison across models.Qwen3-30B-A3B-Think, as a reasoning-specialized model, exhibits a head distribution pattern largely consistent with Qwen3-Coder-30B-A3B: only a relatively small subset of heads shows strong retrieval ability, and these heads are likewise concentrated in the later layers. This suggests that head specialization is a general intrinsic property of pretrained LLMs, rather than an artifact of a specific task or model architecture.

A.2 Diversity of Sparsity between Retrieval Heads.

While Section A.1 characterizes the binary distinction between retrieval and local heads, we further analyze the token-level sparsity patterns within the retrieval head set $\mathcal{H}_{\text{ret}}$. Table 7 reports the number of tokens recalled by top-p ($p = 0.9$) for three representative retrieval heads under different sequence lengths. The results reveal substantial diversity in sparsity behavior across individual retrieval heads:

- *Retrieval heads differ drastically in their token budgets.* Even under the same top-p threshold and the same input sequence, different retrieval heads retain very different numbers of active tokens. For instance, at 64K context length, `L43H31` retains only 21 tokens to cover 90% of attention mass, while `L24H25` requires 24,621 tokens to reach the same coverage threshold—a gap of over three orders of magnitude. This confirms that a single fixed top-k budget cannot simultaneously serve all retrieval heads: it would either over-retain tokens for highly concentrated heads or under-retain for diffuse ones.
- *Head-level sparsity patterns are consistent across context lengths.* Despite the absolute token counts scaling with sequence length, the relative ordering of heads by sparsity remains stable. `L43H31` consistently retains far fewer tokens than `L2H19` and `L24H25` across all tested lengths (32K, 64K, and 128K), suggesting that each head's tendency toward concentrated or diffuse retrieval is an intrinsic property of that head rather than a transient artifact of a specific input.
- *Sparsity diversity motivates per-head dynamic thresholding.* The stark contrast between heads such as `L43H31` (highly concentrated) and `L24H25` (broadly diffuse) highlights why our top-p mechanism is essential. By applying an independent, query-aware threshold to each head, RTPurbo naturally accommodates this diversity: concentrated heads are served with minimal token budgets, while diffuse heads expand their active sets only when necessary. This per-head adaptivity is precisely what static top-k methods fundamentally cannot provide.

Retrieval Head	32K	64K	128K
L2 H19	4540	10468	9858
L24 H25	13836	24621	39614
L43 H31	55	21	42

TABLE 7 Number of recalled tokens selected by top-p ($p = 0.9$) for representative retrieval heads under different sequence lengths.

B Ablations on RTPurbo Design Choices

We conduct ablation studies on several key design settings in RTPurbo.

B.1 Ablation on Retrieval Head Ratio

We compare two retrieval-head ratios in RTPurbo, namely 15% and 30%, on representative subcategories from MMLU-PRO and RULER.

MMLU-PRO Subcategory	RTPurbo (15%)	RTPurbo (30%)
Biology	86.0	85.8
Computer Science	76.8	76.8
History	56.1	56.1
Math	88.2	88.2
Philosophy	59.1	59.0

TABLE 8 Ablation on retrieval-head ratio over MMLU-PRO subcategories.

RULER 64K	RTPurbo (15%)	RTPurbo (30%)
FWE	81.7	81.4
HotPot	62.8	62.6
multi-Q	99.7	99.7
multi-K	98.8	98.6
niah-S	99.9	99.9

TABLE 9 Ablation on retrieval-head ratio over RULER sub-benchmarks.

Benchmark	15%	10%
MMLU-PRO Math	88.2	79.3
MMLU-PRO CS	76.8	70.2
RULER multi-K	98.8	97.4

TABLE 10 Benchmarks with 15% vs. 10% retrieval-head ratios.

The results in Tables 8, 9, and 10 reveal a clear trade-off across different retrieval-head ratios. Increasing the ratio from 15% to 30% brings almost no accuracy improvement on either MMLU-PRO or RULER, while it directly reduces the overall sparsity and increases the training cost, since the first stage must optimize roughly twice as many low-dimensional projection parameters. In contrast, reducing the ratio further to 10% causes a substantial accuracy drop on several representative benchmarks, indicating that the number of retrieval heads becomes insufficient to preserve robust long-range recall. Therefore, for the model used in our experiments, namely Qwen3-30B-A3B, a 15% retrieval-head ratio

provides the best balance between sparsity, training cost, and accuracy, and is thus the most practical design choice.

B.2 Ablation on Low-dimension Size

Benchmark	dim=4	dim=16	dim=32
MMLU-PRO Math	89.1	88.2	88.2
MMLU-PRO CS	76.9	76.8	76.8
RULER niah-S	100	99.9	99.9
RULER HotPot	63.0	62.8	62.7

TABLE 11 Ablation on low-dimension size over representative benchmarks.

End-to-end Accuracy. We compare different low-dimensional sizes in RTPurbo and study how the projection dimension affects both end-to-end task accuracy and the fitting quality of the low-dimensional relevance space. Specifically, we evaluate three representative settings, namely dim = 4, dim = 16, and dim = 32, on a subset of representative benchmarks. Table 11 summarizes the end-to-end accuracy results. We observe that dim = 16 and dim = 32 deliver nearly identical accuracy across all evaluated benchmarks. Since increasing the dimension from 16 to 32 doubles the number of trainable low-dimensional projection parameters, this result suggests that dim = 16 is already sufficient to capture the attention distribution of retrieval heads accurately, and that further enlarging the projection space brings little practical benefit.

Interestingly, dim = 4 yields the highest end-to-end accuracy in this comparison. However, this should not be interpreted as evidence that a smaller low-dimensional space provides a better approximation. Instead, Table 12 shows that dim = 4 has substantially weaker fitting ability, which causes the top-p selector to recall many more tokens in order to cover the same amount of attention mass. As a result, the actual sparsity becomes much lower under dim = 4, and the corresponding accuracy improves because the sparse model behaves more similarly to a less aggressively pruned model.

Fitting Ability of Low-dimension Space. Table 12 reports the number of recalled tokens for retrieval head `L24H25` under different projection dimensions when using top-p with $p = 0.9$. The results show a clear trade-off between expressiveness and sparsity. When the projection dimension is too small, the compressed relevance space cannot faithfully model the full attention distribution, forcing the selector to retain substantially more tokens in order to recover the same amount of attention mass. This is exactly what happens at dimension 4, where the recalled token count rises sharply across all sequence lengths, indicating poor fitting ability and weak sparsity. Increasing the dimension to 16 dramatically improves the quality of the approximation and yields the smallest recalled-token budget overall, showing that this setting best captures the retrieval structure of the head while preserving

high sparsity. Further increasing the dimension to 32 does not bring additional benefit; instead, it consistently requires more recalled tokens than dimension 16. This suggests that a moderately compact subspace is sufficient, and that a larger projection dimension may introduce unnecessary flexibility without improving token selection quality. Therefore, we choose dimension 16 as the default setting, since it achieves the best balance between fitting accuracy, sparsity, and computational efficiency.

Dimension	32K	64K	128K
4	25256	45280	89081
16	13771	25725	49133
32	15526	28464	54727

TABLE 12 Number of recalled tokens for retrieval head L24H25 under top-p ($p = 0.9$) with different input lengths and projection dimensions.

C Details of the Two-Stage Training Pipeline

In this section, we present the detailed procedure of our two-stage training pipeline and analyze the overall training workflow for Qwen3-Coder-30B-A3B.

C.1 Details of Low-dimension Projection Training

For Qwen3-Coder-30B-A3B, the total number of query heads is 1536, with head dimension $d_h = 128$. After offline calibration, we select the top 210 heads with the highest retrieval scores as retrieval heads, corresponding to roughly 15% of all heads. For stage-1 training, we construct the training set by sampling 8,000 sequences from the open-source FineWeb dataset, each with a length between 32K and 80K tokens. For each retrieval head, our objective is to align the original attention-score distribution with the compressed attention-score distribution, while training only the low-dimensional query/key projection parameters. Under our setting, each head introduces $2 \times 128 \times 16 = 4096$ trainable parameters, corresponding to the query and key projections with head dimension 128 and low dimension 16. Therefore, the total number of trainable parameters in stage 1 is $210 \times 4096 \approx 8.6 \times 10^5$, i.e., about 840K parameters in total.

The training settings are summarized in Table 13. Specifically, the learning rate is linearly warmed up from 0 to the peak value of 10^{-3} over the first 100 steps, and then decayed with a cosine annealing schedule. As shown in Figure 9a, the training loss of Layer 24 Head 25 already converges well within about 600 steps, and we observe highly similar convergence behavior for the other retrieval heads. Since each training sequence contains 48K tokens on average, the total token budget of this stage is approximately $48\text{K} \times 600 \simeq 30\text{M}$ tokens.

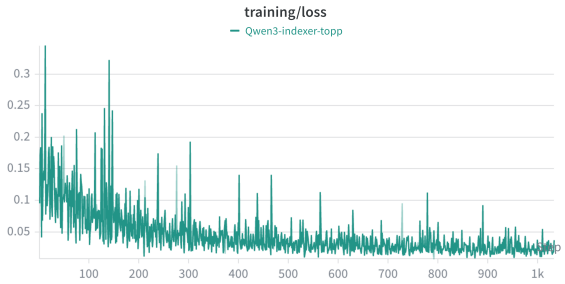
TABLE 13 Stage-1 training configuration.

Training Settings	
max-lr	1e-3
warmup-steps	100
lr-scheduler	consine
weight-decay	0.01
max-grad-norm	1.0

TABLE 14 Stage-2 training configuration.

Training Settings	
max-lr	3e-6
warmup-steps	200
lr-scheduler	constant
weight-decay	0.01
max-grad-norm	1.0
global-batch-size	8
micro-batch-size	1

(a) Training loss curve of retrieval head L24H25 during stage-1 low-dimension projection training.



(b) End-to-end training loss curve during stage-2 self-distillation.

FIGURE 9 Training loss curves of the two-stage training pipeline.

C.2 Details of End-to-End Self-distillation Training

For stage-2 training, we sample 8,000 long reasoning examples in dialogue format from Dolma 3 Longmimo Mix. Each sequence is longer than 32K tokens, the average sequence length is about 48K tokens, and the average number of training label tokens is about 300. We first run a forward pass on the original model over the entire training set, extract the top-10 next-token prediction logits at each position, and cache them as distillation targets. During training, we attach the low-dimensional parameters learned in stage 1 to the selected retrieval heads and keep these parameters frozen, while performing end-to-end optimization over the model weights. The objective of this stage is to align the logits of the sparse model with those of the original model before sparsification.

The training settings are summarized in Table 14. In particular, we use a relatively small learning rate to avoid drifting away from the model's original capabilities during end-to-end finetuning. As shown in Figure 9b, the training loss of stage 2 converges within about 600 steps. The full training corpus contains roughly 180M tokens in total, while the actual

number of label tokens involved in learning is only about 1.2M.

D Limitation

Although RTPurbo achieves strong efficiency–accuracy trade-offs, it still has several limitations.

- *Dependence on stable head specialization.* Our method relies on the empirical observation that attention heads can be partitioned into retrieval and local groups through offline calibration. While this behavior is stable in the models we study, the quality of this partition may degrade for models with weaker head specialization or under substantial domain shift.
- *Incomplete sparsification and limited evaluation scope.* In the current design, retrieval heads still use full dense attention during prefill. In addition, our experiments mainly focus on the Qwen3 family and on long-context and reasoning workloads, so broader validation on other architectures and domains is still needed.

We expect these limitations to be addressed by future work on more adaptive head routing, stronger prefill sparsification, and broader cross-model evaluation.